CHAPTER 8

ARRAYS

Definition:

An array is a contiguous block of memory that stores multiple values of the same data type under a single variable name.

- It can be considered as a collection of variables stored at continuous memory blocks.
- Each value in the array can be accessed using its index.

Why Do We Need Arrays?

- To store multiple values of the same type using a single variable.
- Reduces code complexity and increases efficiency.
- Suitable for storing large datasets like student marks, salaries, etc.

Characteristics of Arrays

1. Fixed Size:
   - Size must be specified at the time of declaration.
   - Cannot be increased or decreased dynamically.
2. Homogeneous Elements:
   - Stores elements of the same data type (int, float, double, etc.).
3. Contiguous Memory Allocation:
   - Elements are stored in consecutive memory locations.
4. Indexing:
   - Index starts from 0 up to (size - 1).

ARRAY vs VARIABLE

Feature	Variable	Array
Storage	Single value	Multiple values
Naming	Named memory block	Not individually named
Access	Direct	Via index

Disadvantages of Arrays

- Fixed Size: Cannot grow or shrink after declaration.
- Same Data Type: Can't store multiple types in one array.
- Only Stores Values: Cannot store objects with different types without casting or using Object arrays.

Array Declaration and Initialization

Declaration:

```
1  int[] a;   // Preferred syntax
2  int a[];   // Also valid
3
```

Initialization:

```
1  class ExampleArrayInitialization
2  {
3      public static void main(String [] args)
4      {
5          // INITIALIZE ONLY SIZE USING NEW KEYWORD, LATER ADD ELEMENTS
6          int a[] = new int[4];
7          // DIRECTLY INITIALIZE WITH VALUES
8          int a[] = {12, 5, 87, 6};
9      }
10 }
11
```

Important Array Properties

Property	Description
length	Returns the size of the array
Index range	Starts at 0 and ends at length - 1
Default values	int → 0, boolean → false, String → null, etc.

Accessing Array Elements

System.out.println(a[2]); // prints 3rd element

Accessing involves:

- Array reference (name of array)
- Index (position of element in square brackets)

Array Example

```
1  class ExampleArray
2  {
3      public static void main(String [] args)
4      {
5          int a[] = new int[4];
6          a[0] = 14;
7          a[1] = 12;
8          a[2] = 8;
9          a[3] = 16;
10         System.out.println(a[2]);
11         System.out.println(Arrays.toString(a));
12     }
13 }
```

Real-life Analogy

Think of an array like an apartment building:

- Each flat = an element
- The building address = base address
- Flat number = index

Access a flat using:

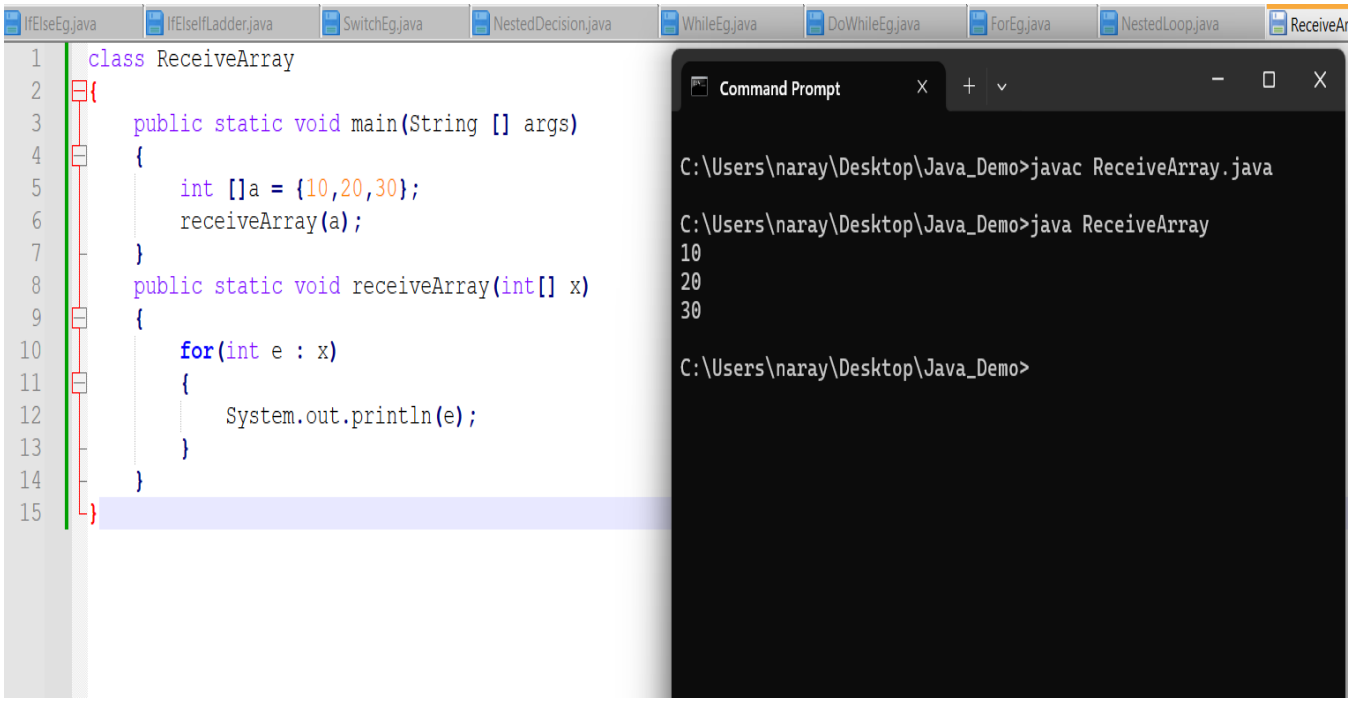
BuildingAddress + FlatNumber → a[3] (4th flat)

TEN POINTS TO REMEMBER:

1. Continuous memory blocks
2. Stores homogeneous elements
3. Declare an Array example: int a[];
4. Instantiate an Array example: new int[7];
5. Initialize an Array examples:
   int a[] = new int[4];
   int a[] = {54,87,12,5};
6. Size is mandatory
7. Indexing is mandatory {starts at 0 and ends at (a.length)-1}
8. The elements are initialized with default values
9. Re-initialization of elements example: a[0] = 41, a[1] = 8
10. CRUD operations are performed using index and complex logic

ARRAY WITH METHODS

Method to Receive an Array:



Method to Return an Array: